Evaluating Coding Agent Quality

Application to Software Design Principles

Akram Ben Fekih

ISSAT Sousse · 2025/2026

Do autonomous coding agents respect **SOLID** design principles?

4,245 Java pull requests · 1,010 repositories · 3 agents

The benchmark–reality gap

What we can already measure

- Agents now open pull requests on real repositories, unattended.
- Benchmarks answer one question: **do the tests pass?** (HumanEval, SWE-bench).

What nobody measures

- Whether the code is *structured* so the next person can change it.
- Code can be functionally correct and still be tightly coupled, poorly factored and expensive to extend.

The gap

Functional correctness is a **floor**, not a design claim.

A merged pull request tells you a maintainer accepted it. It does not tell you what it did to the architecture.

And most studies look only at *merged* code, which measures what reviewers accepted rather than what the agent wrote.

Research questions

RQ1 Do the three agents differ in SOLID adherence, and does the outcome of review (merged / rejected / open) relate to it?

RQ2 **Can SOLID be measured at scale, and how faithfully?**
This is the question the thesis takes most seriously: a proxy is only worth reporting if its fidelity is measured rather than assumed.

RQ3 Where does the risk sit, *inside* a unit or *between* units?

4,245	1,010	1,495	3
analysable Java PRs	repositories	judged entities	coding agents

Five principles, five deterministic proxies

	The forbidden smell	What is measured	Scope
SRP	god class	seven-metric rubric + conceptual-cohesion suppressor	class
OCP	type/tag switch	dispatch-chain width + enum suppressor	dispatch chain
LSP	non-substitutable override	the three faults `javac` cannot catch	override pair
ISP	fat interface	contract width + implementer stub ratio	contract
DIP	concretion dependency	newly cyclic module pairs	pull request

Why Liskov is only three signals in Java

`javac` already rejects a changed parameter list (an *overload*), a narrowed parameter type, and reduced visibility. Code that compiles cannot contain those faults, so the proxy targets only what the compiler cannot see: **refused bequest**, **strengthened precondition**, **weakened postcondition**.

The corpus, and why rejected code is in it

Agent	Analysable PRs	Mean files
Claude Code	1,109	46.4
GitHub Copilot	2,366	18.5
OpenAI Codex	770	10.9
Total	**4,245**	

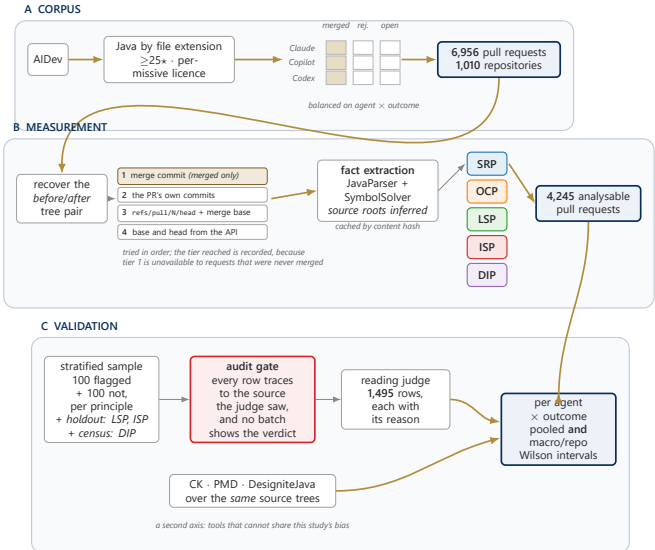
- 6,956 curated → 4,245 analysable; every drop is accounted for.
- Merged, rejected **and** still-open requests.

A corpus of merged code is survivorship-biased

It measures **what maintainers accepted**, not what the agent produced. Including rejected and open requests makes the review filter itself observable.

Caveat, stated throughout: "rejected" is mostly `closed_unreviewed`, that is superseded or stale, not a maintainer's objection on quality.

The measurement pipeline



The instrument has to be validated, not assumed

Two independent axes

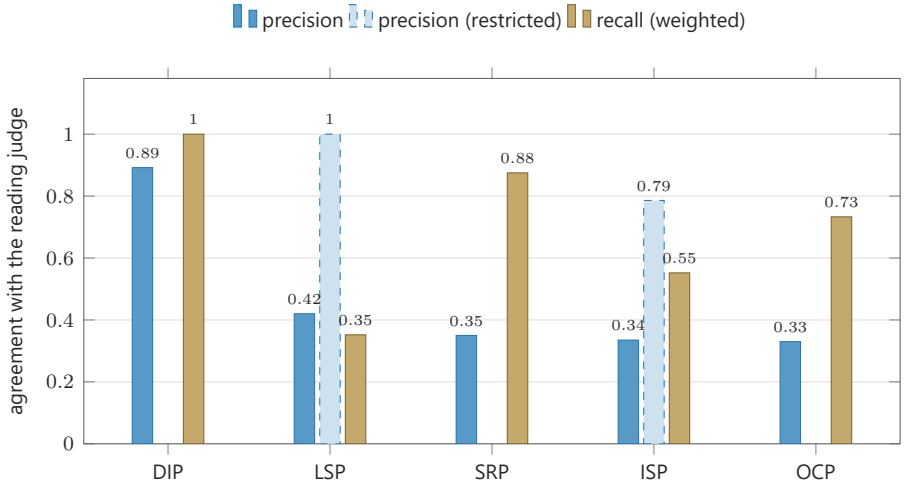
1. A **reading judge** labels 1,495 entities, 100 flagged and 100 unflagged per principle, and records a reason for every verdict.
2. **CK, PMD and DesigniteJava** run over the *same* source trees, because a judge can be wrong in the same direction as the rubric.

Two guards that make it trustworthy

Both strata are sampled. A sample drawn only from flagged rows can measure precision but can *never* discover a violation the detector missed.

An audit gate. Every judged row must trace back to the source file the judge was shown, and no batch may reveal the detector's own answer.

RQ2: what the instrument actually does



Precision below 0.5 for four principles is **the finding**, not a defect: a structural proxy asks a syntactic question and the principles are semantic. The gap is what these numbers quantify.

The holdout: two refinements, opposite fates

Two quantities were *fitted* to the first sample. Scoring them on that same sample reports optimism, so both were re-scored on **200 fresh entities each**, excluding every previously judged row.

Refinement	Fitted sample	Holdout	Pooled
LSP restricted to *refused bequest*	1.000 (31/31)	**1.000** (33/33)	1.000 (64/64)
ISP restricted to $\geq$30 members	0.862 (25/29)	**0.704** (19/27)	0.786 (44/56)

Why they came apart

Refused bequest is categorical: a body that is nothing but a `throw` either is or is not that shape. There is nothing to tune.

"30 members" is a numeric threshold read off the false positives it was then scored against, so it absorbed some of that sample's noise. Its fitted value lies *outside* the holdout's own interval (0.515–0.841).

Recall must be weighted, and that exposed a blind spot

Each sample is half flagged, half unflagged. **The corpus is not.** For Liskov: 1,546 flagged override pairs against **79,689** unflagged, so one sampled unflagged row stands for ≈797 entities and a flagged one for 15.

	raw	weighted
SRP	0.972	0.875
OCP	0.943	0.733
ISP	0.914	0.552
DIP	1.000	1.000
LSP	0.966	**0.352**

The three Liskov violations the detector missed

All three are **silent** refusals, not declared ones:

- a Vaadin root node throws for *every* non-null parent its base accepts;
- a CommCare controller overrides a method that disables a login button with an **empty body**;
- an nd4j operation implements `setY` as `this.y = null`, a setter that discards its argument.

Flag rates bound SRP, OCP and DIP, *not* LSP and ISP.

Second axis: agreement with published tools

150 repositories, **111,029** shared production types.
No tool is treated as ground truth.

Pair (single-responsibility)	κ
ours ~ PMD `GodClass`	**0.574**
ours ~ DesigniteJava	**0.531**
PMD ~ DesigniteJava	0.384

Rank correlation with CK: **wmc 0.977**, methods 0.969, loc 0.944.

The result that matters

The two *established* tools agree with each other **less** ($\kappa = 0.384$) than either agrees with the rubric built here.

68.0% of our flags (4,304 of 6,326) are corroborated by at least one independent tool.

Cohesion is the exception (0.288), but CK's own two cohesion variants agree only at 0.490 with each other, so the disagreement is in the concept rather than in our implementation.

RQ1: flag rate by agent

Principle	Claude Code	GitHub Copilot	OpenAI Codex
SRP (authored classes)	9.5% (8.4%)	**5.1%** (3.5%)	6.8% (7.4%)
OCP	10.6% (10.1%)	6.4% (3.0%)	**2.1%** (2.2%)
ISP	8.7% (6.0%)	6.7% (4.0%)	**2.9%** (5.0%)
DIP	6.5% (5.1%)	4.1% (2.8%)	**3.2%** (5.4%)
LSP	0.7% (0.3%)	0.3% (0.1%)	**0.0%** (0.0%)

Pooled rate first, macro average over repositories in brackets, so one large repository cannot report its own conventions as an agent's behaviour.

Three qualifications, all stated in the thesis

Era. Median PR dates are seven months apart (Codex Aug 2025, Copilot Jan 2026, Claude Code Mar 2026). Agent and era are confounded.
Thin cells. Codex contributes 770 PRs, so its intervals are wide.
Change size. See the next slide, this one is decisive.

The ordering is conditional on change size

The three agents do not write changes of the same size:

	mean Java files touched
Claude Code	46.4
GitHub Copilot	18.5
OpenAI Codex	10.9

A delta-level detector has more opportunity to fire when more code changes, so a larger change accumulates more flags without being worse code.

What survives the control

Within size bands, the pooled ordering holds **only for the largest band** (11+ files). In the smallest band **Copilot** leads on OCP and ISP and **Codex** leads on DIP.

Never quote the per-agent table without this qualification.

RQ1: does review select on SOLID adherence?

Tier-matched subset, 1,521 merged against 1,177 rejected pull requests, because tier 1 reconstruction is available only to merged requests.

	Merged	Rejected	Diff.
OCP	8.3%	8.0%	-0.4 pp
ISP	7.5%	6.6%	-0.9 pp
DIP	4.6%	6.6%	$+2.0$ pp
LSP	0.5%	0.5%	± 0.0 pp

Merged and rejected code is structurally indistinguishable

No consistent difference, and the one that is largest points the "wrong" way. **Human review does not appear to select on these properties.**

A methodological consequence: for these signals a merged-only corpus is not a biased sample of agent output. That could not have been established without measuring the requests maintainers turned down.

RQ3: risk is intra-unit, not inter-unit

Principle	Flagged entities present	Est. genuine	PRs that introduced one
SRP	11,822 classes	≈4,100	n/a
OCP	2,577 dispatch chains	≈850	286 (2.2%)
ISP	1,090 contracts	≈350	276 (2.1%)
LSP	681 override pairs	≈270	**16** (0.4%)
DIP	195 cycles	≈175	195 (4.1%)

Both columns corrected for measured precision.

The structural explanation

An **intra-unit** smell (SRP, OCP) lives inside one class, so a single pull request can author it. An **inter-unit** smell (LSP, ISP, DIP) exists only in a *relation* between two units, and one change rarely opens both ends. 681 refused-bequest relationships exist in the code the agents touched; they introduced **16**.

Answering the research questions

RQ1 The agents differ, Claude Code highest on all five and Codex lowest on four, **but the ordering is conditional on change size** and agent is confounded with era. Review does not select on these properties.

RQ2 SOLID *can* be measured at scale, with an operating characteristic that must be stated rather than collapsed into one number. Precision 0.330–0.892; weighted recall 0.352–1.000. Two detectors are repairable from their own labels, and only the **categorical** repair survived a holdout.

RQ3 Risk is **intra-unit**. Agents author god classes and type switches; they almost never author a broken subtype relation, because that takes two coordinated changes.

Contributions

1. A **benchmark over real, attributable units**: 4,245 Java pull requests across 1,010 repositories, with merged, rejected and open requests alike.
2. **All five SOLID principles** operationalised as deterministic proxies, each with *measured* construct validity rather than assumed proxy fidelity.
3. An **out-of-sample holdout** that separates a refinement which generalises from one that was partly fitted, and a demonstration that stratified recall must be weighted before it means anything.
4. The **intra/inter-unit distinction** as an empirical result: per-change measurement systematically cannot see inter-unit damage.

Limitations

- **Agreement is rubric versus reading judge.** LLM judging of SOLID is a validated technique (Pehlivan et al., 2025), but the judge can err in the same direction as the rubric. The external-tool axis is the defence, and it covers SRP only, since no published source-only Java tool has an OCP or ISP rule.
- **The judge and one agent are both Claude.** Tested directly: on four of five principles Claude Code's flags are confirmed *more* often, which is the opposite of leniency.
- **"Rejected" mostly means superseded**, not refused on quality.
- **Source-only analysis** cannot resolve supertypes declared outside the repository: median resolution 87.0%, so LSP and ISP under-count in framework-heavy projects. The direction is favourable, an unresolved supertype removes a candidate rather than inventing one.
- **Weighted recall for LSP and ISP rests on few observations**, so its intervals are wide.
- **Java only.** Nothing here is claimed for other languages.

Future work

- **A churn-based Open/Closed signal.** Openness is uncorrelated with dispatch width, so the obvious threshold is the wrong instrument; a class edited across many pull requests is by definition not closed.
- **A closed-set suppressor that recognises external specifications** (JDBC type codes, protobuf scalars, Unicode blocks), which are closed in fact but open to the compiler.
- **Narrow the recall intervals** for the two blind detectors with a sample drawn only from unflagged entities, aimed at *silent* refusals.
- **A second language** with the same agent coverage: the framework is language-parameterised, only fact extraction is language-specific.

Thank you

The instrument is reported with its error profile, not without one.

Every number in this deck is recomputed
from the data by `verify_thesis_numbers.py`.

Questions